Práctica 4: Automatización con generación de código

Presentado por Sara V. Guerrero Espinosa
28 de marzo de 2026

Índice:

Preparación del entorno..... 1
Generación automática de código de visualización con IA.....3
Visualización geoespacial de la distribución de la renta..... 9
Orquestación reactiva mediante sensor basado en eventos..... 12
Control de versiones e integración continua.....13

En la presente práctica se abordó el diseño e implementación de un pipeline de datos orientado a la generación de visualizaciones en el marco de un enfoque DataOps. El objetivo principal consistió en integrar un servicio de IA generativa dentro de un flujo orquestado con Dagster, de manera que fuese posible construir, validar y ejecutar código de visualización de forma automatizada a partir de descripciones basadas en la gramática de gráficos. Asimismo, se buscó incorporar procesos de despliegue automático en GitHub, explorar la representación geoespacial de los datos de renta en Canarias, e implementar un sistema de orquestación reactiva basado en sensores, con el fin de automatizar la actualización de resultados ante cambios en los datos de entrada.

Todos los ficheros de trabajo, incluídos los datos utilizados, los códigos de Python, las visualizaciones generadas y este mismo informe se encuentran subidos al repositorio de Github dispuesto en el siguiente link: [Práctica 04 Visualización - GitHub Sara Guerrero]

A continuación se detalla la solución de los diferentes apartados de la práctica:

Preparación del entorno

En primer lugar, se verificó el correcto funcionamiento del servicio de inteligencia artificial proporcionado por la universidad. Para ello, se desarrolló un script de prueba en Python (*test_ia.py*) que realizaba una petición HTTP al endpoint del modelo, enviando un mensaje sencillo y comprobando tanto el código de estado como la respuesta generada.

```python
import requests

url = "http://gpu1.esit.ull.es:4000/v1/chat/completions"

headers = {
    "Content-Type": "application/json",
    "Authorization": "Bearer sk-1234"
}

data = {
    "model": "ollama/llama3.1:8b",
    "messages": [
        {"role": "user", "content": "Hola. Dime para qué sirve un firewall"}
    ]
}

response = requests.post(url, headers=headers, json=data)

print(response.status_code)
print(response.text)
```

La respuesta obtenida presentó un código de estado 200 junto con un contenido coherente generado por la IA, lo que permitió confirmar que el servicio se encontraba operativo.

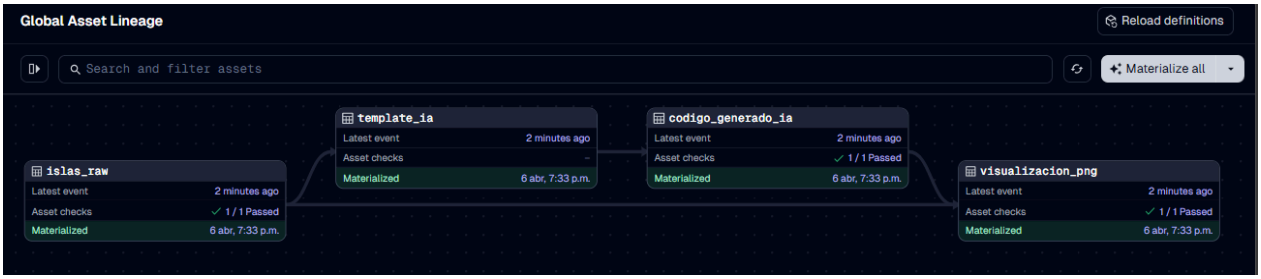
```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS

PS C:\Users\Usuario\Documents\Aaa ULL\Visualización\Practicas-Visualizacion-de-datos> & C:\Users\Usuario\AppData\Local\Programs\Python\Python312\python.
exe "c:/Users/Usuario/Documents/Aaa ULL/Visualización/Practicas-Visualizacion-de-datos/Practica_04/data/test_ia.py"
200
{"id":"chatcmpl-892cf027-188b-4507-8b5f-082a232e26be","created":1775495655,"model":"ollama/deepseek-coder:6.7b-instruct-q4_K_M","object":"chat.completio
n","choices":[{"finish_reason":"stop","index":0,"message":{"content":"Un firewall es una red informática que actúa como un sistema de seguridad al permi
tir o denegar el tráfico entrante y saliente en función del origen, destino, protocolo y puerto de las comunicaciones. En términos generales, sirve para
 proteger la información de un entorno digital contra posibles amenazas externas a través de medidas físicas o lógicas que controlan el acceso al sistem
a desde una red interna.\n","role":"assistant"}}],"usage":{"completion_tokens":115,"prompt_tokens":87,"total_tokens":202}}
PS C:\Users\Usuario\Documents\Aaa ULL\Visualización\Practicas-Visualizacion-de-datos>
```

Posteriormente, se ejecutó el fichero *test_prompt.py*, proporcionado por la práctica. Este pipeline incluyó la carga y validación del dataset de renta por isla, la construcción de un prompt estructurado basado en la gramática de gráficos, la generación de código en Plotnine a través de un modelo de lenguaje, y su ejecución dinámica en un entorno controlado para producir una visualización.

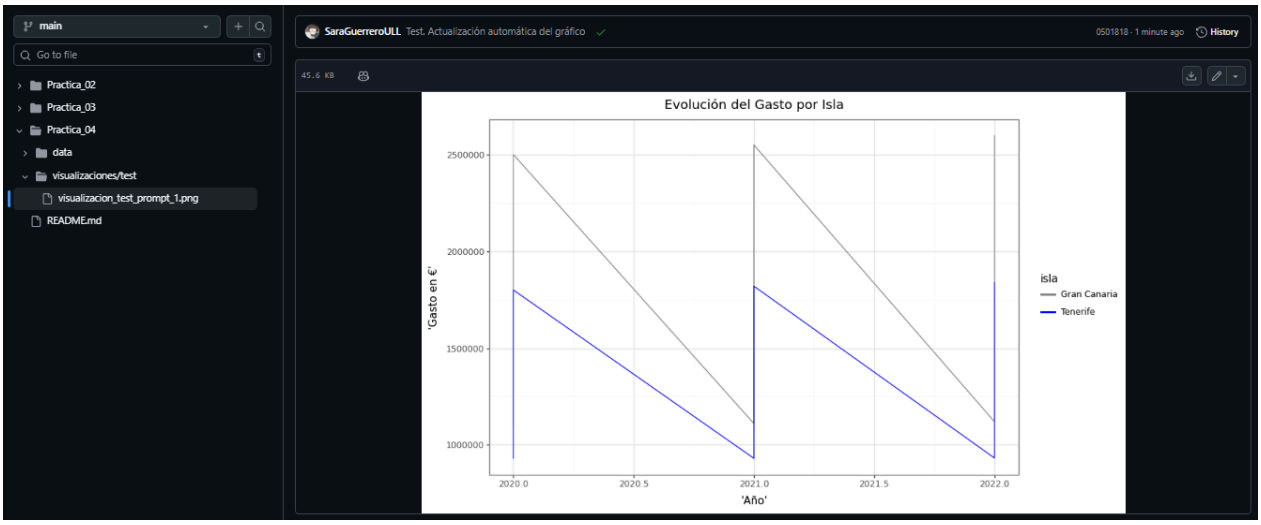
A partir de este punto, se introdujeron diversas modificaciones con el objetivo de mejorar la robustez del sistema. Se corrigieron errores en la limpieza del código generado y se aseguró la eliminación de elementos de markdown y referencias inválidas. Se reforzó la construcción del prompt mediante la incorporación de restricciones explícitas que obligaban a generar exclusivamente código Python ejecutable. Asimismo, se añadieron validaciones sobre el código generado, con comprobaciones específicas sobre la presencia de estructuras esperadas como el uso de ggplot.

En la fase de visualización, se adaptó la gestión de rutas de salida para garantizar el almacenamiento correcto de los archivos generados. Además, se preparó el entorno de ejecución con el fin de evitar errores derivados del uso de exec(). De forma complementaria, se incorporaron checks en Dagster que permitieron validar tanto la calidad del código generado como la correcta creación de las visualizaciones, lo que reforzó el enfoque de control dentro del pipeline.

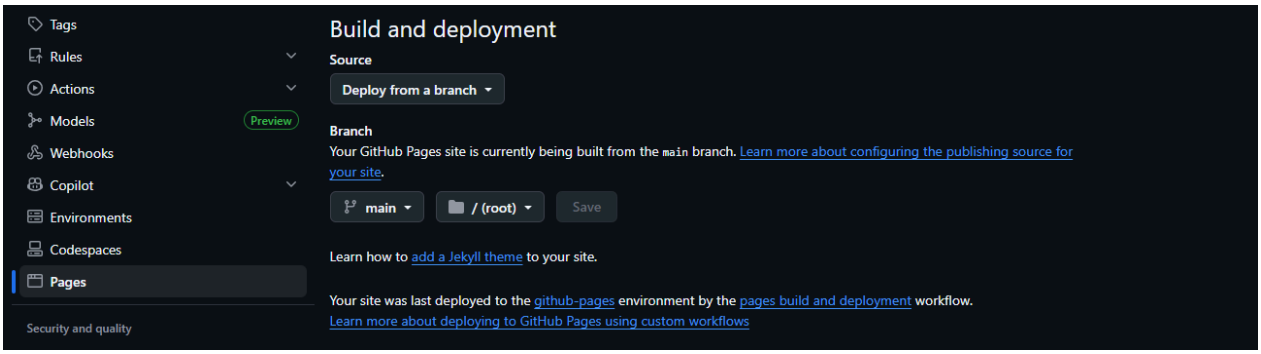
Tras los ajustes realizados, el pipeline se visualizó correctamente en Dagster y su ejecución finalizó sin errores. Todos los assets se materializaron de forma satisfactoria y la visualización se publicó en GitHub.



La visualización obtenida por el pipeline y publicada automáticamente a Github se vio así:



Se vinculó y configuró el repositorio a github pages como se nos indicaba en la guía de la práctica



Finalmente, la visualización anterior quedó expuesta a través de GitHub Pages, a través del siguiente link: GitHub Pages: visualizacion_test_prompt_1.png

Generación automática de código de visualización con IA

Una vez validado el funcionamiento del entorno, se retomó el proyecto de análisis de la distribución de la renta. En esta fase, el objetivo consistió en automatizar la generación de visualizaciones, de modo que el código necesario para construir los gráficos no se escribiera manualmente, sino que fuera producido por un modelo de inteligencia artificial a partir de descripciones estructuradas.

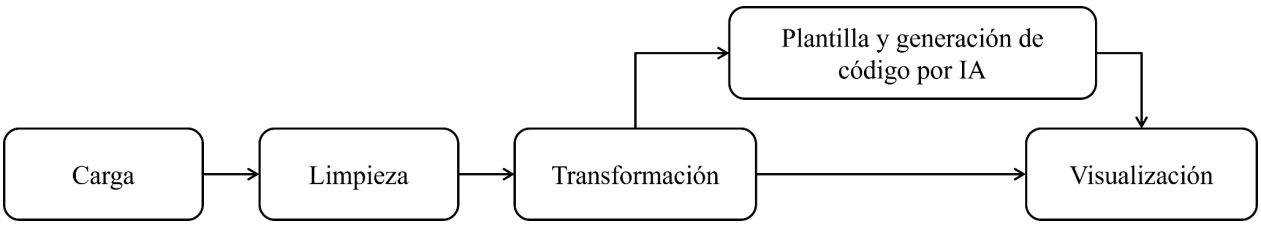
El pipeline se organizó en cinco fases principales: carga de datos, limpieza, transformación e integración, generación de código mediante plantillas de prompt y, finalmente, visualización. Esta estructura permitió separar claramente la lógica de preparación de datos de la capa de generación y representación gráfica.

La generación automática de gráficos se basó en una arquitectura de prompts dividida en dos componentes: system content y user content. El system content definió el comportamiento general del modelo, estableciendo restricciones estrictas sobre el formato de salida. Se indicó que la respuesta debía consistir exclusivamente en código Python ejecutable, sin explicaciones adicionales ni sentencias de importación, y que debía ajustarse a la gramática de gráficos. Por su parte, el user content describió de forma específica el gráfico a construir, incluyendo las variables implicadas, el tipo de geometría, los mapeos estéticos y ciertos aspectos de diseño. Esta separación permitió mantener un equilibrio entre control y flexibilidad en la generación.

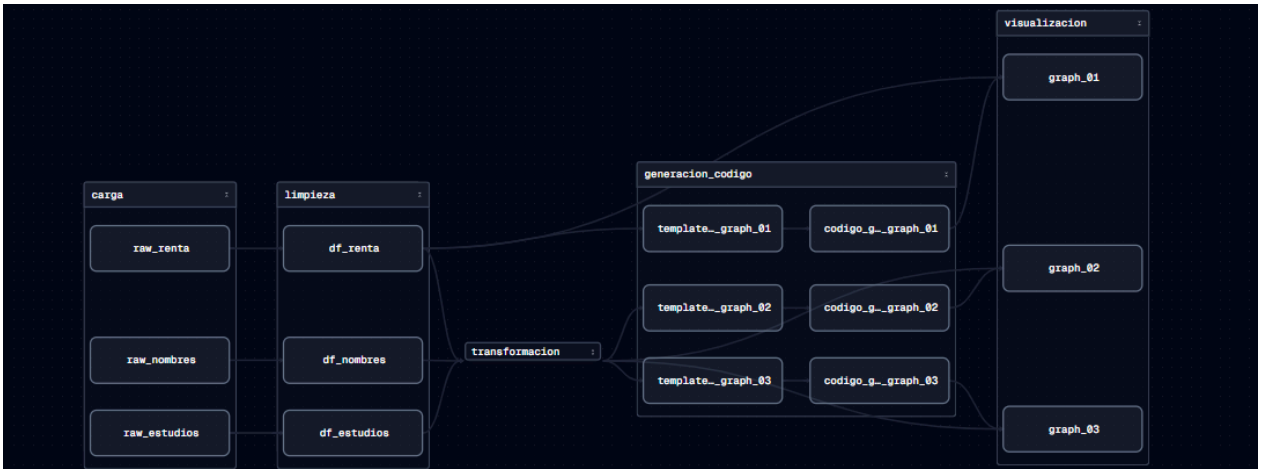
Dentro del pipeline, el proceso se implementó mediante varios assets: El asset del tipo *template_ia* construía el payload que se enviaba al modelo, integrando la información del dataset, como nombres de columnas y categorías disponibles, con las instrucciones definidas en los prompts. Este asset devolvía una estructura que incluía el modelo a utilizar, los mensajes y parámetros como la temperatura. A continuación, el asset del tipo *codigo_generado_ia* realizaba la petición al servicio de IA y procesaba la respuesta obtenida. En esta fase se aplicó una limpieza del contenido generado, para eliminar posibles artefactos de formato y asegurar que el resultado fuera un bloque de código válido.

En la capa de visualización, el código generado se ejecutó dinámicamente mediante exec() dentro de un entorno controlado. Este paso permitió construir el objeto plot de Plotnine sin intervención manual. Posteriormente, el gráfico se exportó como imagen, se almacenó localmente y se publicó automáticamente en el repositorio.

Las fases del pipeline se conectaban así:



En Dagster, la representación del pipeline reflejó estas capas:



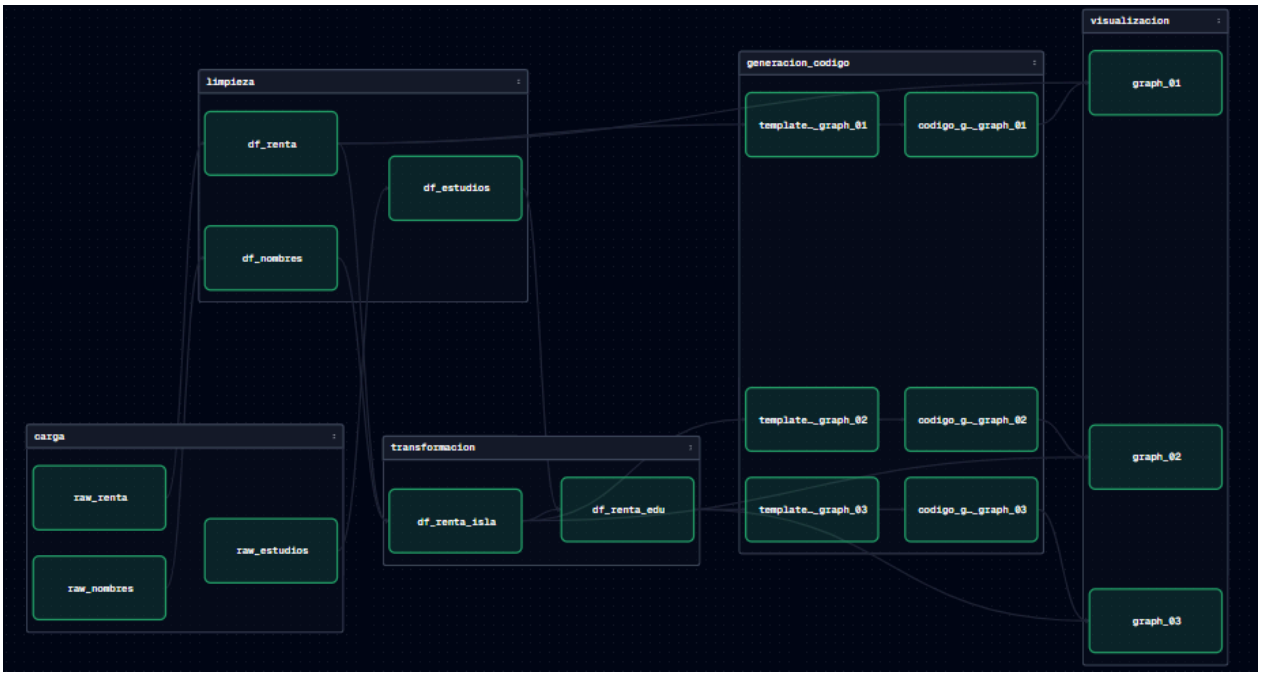
El diseño de los prompts combinó descripciones en lenguaje natural con restricciones técnicas explícitas con el fin de guiar a la IA hacia una generación de código coherente con la gramática de gráficos y las buenas prácticas de visualización. En primer lugar, el gráfico se definió en términos conceptuales, especificando qué se deseaba representar, qué variables intervenían y cómo se relacionaban. Esta formulación permitió estructurar adecuadamente las capas del gráfico, incluyendo mapeos estéticos, geometrías y posibles facetas.

En paralelo, se incorporaron “anclas técnicas” que actuaron como mecanismos de control sobre la salida generada. Entre ellas, se impuso el uso obligatorio de elementos como ggplot, geom_col, scale_fill_manual o theme_bw(). Estas restricciones redujeron la ambigüedad del modelo y favorecieron la consistencia visual entre las distintas representaciones.

Asimismo, se establecieron directrices de diseño orientadas a mejorar la legibilidad y coherencia estética de los gráficos. Estas incluyeron la definición de una paleta de colores, la rotación de etiquetas en ejes categóricos, la posición de la leyenda y el formateo de valores en porcentaje. Con ello se buscó evitar errores habituales y asegurar un resultado visual homogéneo.

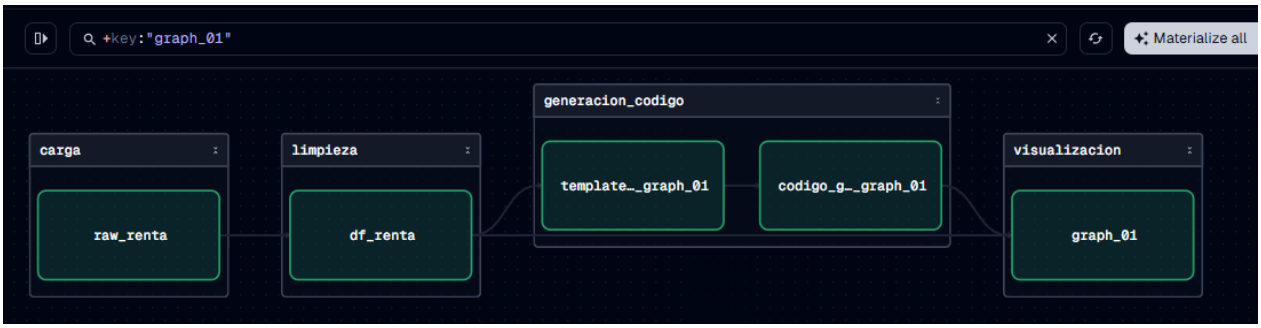
Por último, se definieron restricciones estrictas sobre el formato de salida, en las que se exigió la ausencia de imports, la no inclusión de transformaciones de datos y la generación exclusiva de un bloque de código del tipo plot = (...). Estas condiciones garantizaron la ejecución directa del código dentro del pipeline y permitieron separar de forma clara la lógica de procesamiento de datos de la capa de visualización.

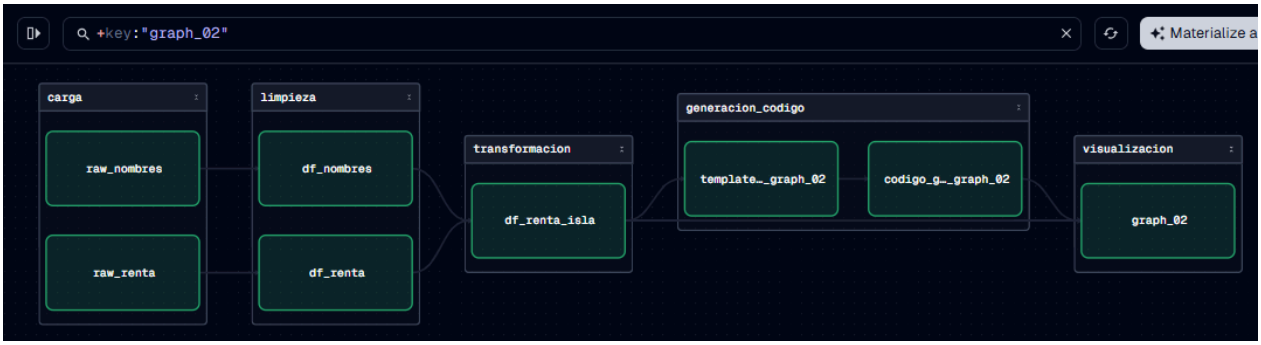
Tras varias iteraciones, se obtuvieron tres visualizaciones distintas a partir del pipeline. El pipeline completo y correctamente materializado y con todos los checks aprobados se vio así:



A medida que aumenta el número de assets, la trazabilidad del flujo completo del pipeline para cada visualización resulta menos evidente. No obstante, Dagster ofrece funcionalidades de

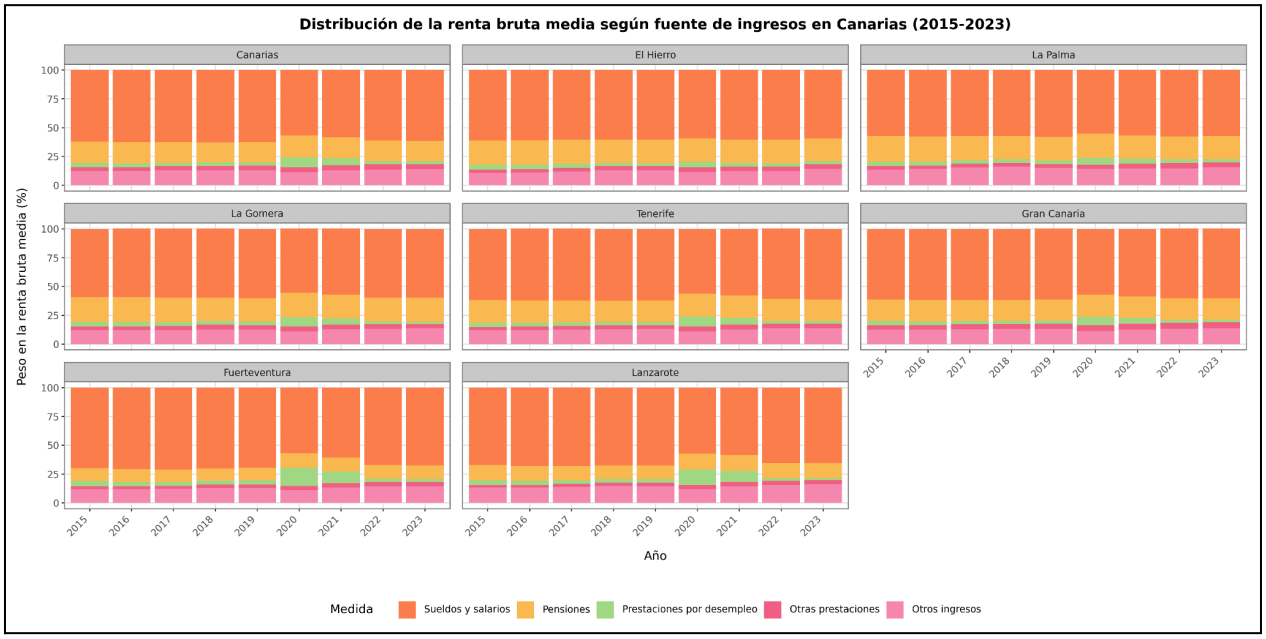
filtrado que permiten aislar los assets relevantes y verificar la secuencia de ejecución de forma precisa. De este modo, fue posible corroborar el flujo específico seguido en cada visualización, el cual se estructuró de la siguiente manera:





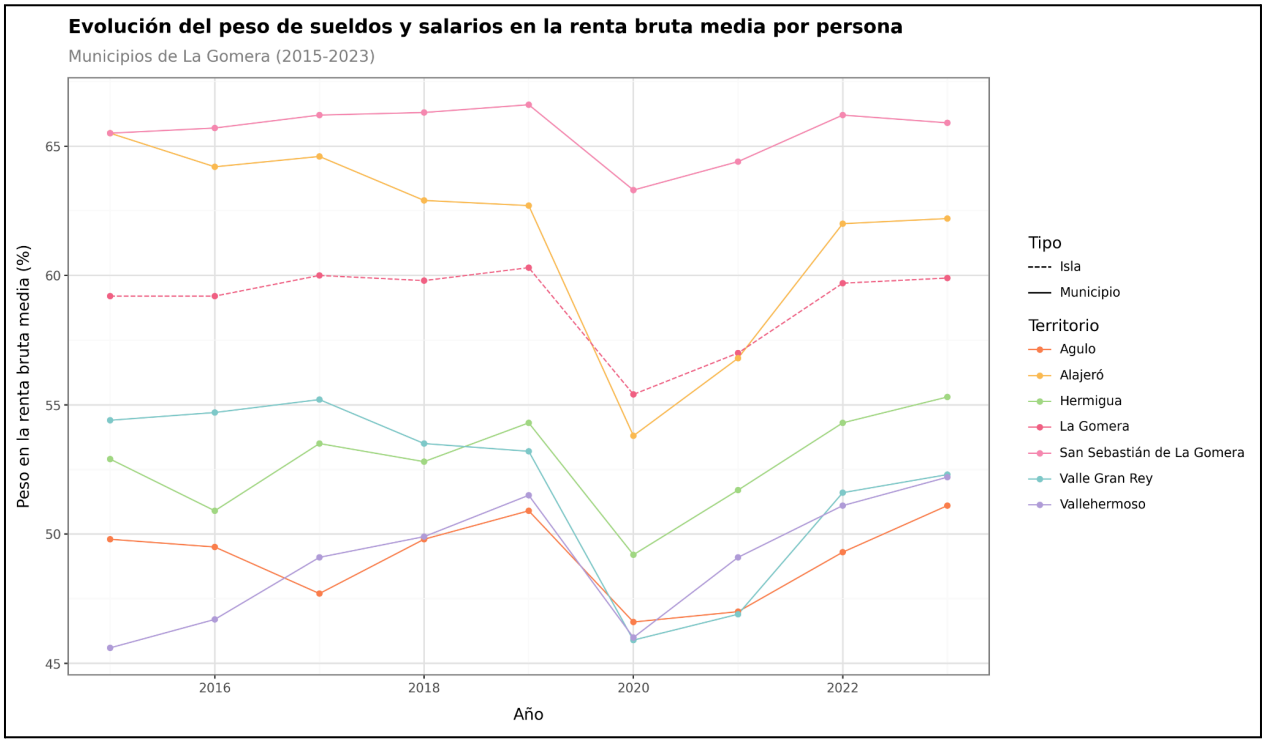
Las visualizaciones generadas fueron correctamente almacenadas y publicadas en GitHub Pages. A continuación, se presentan las gráficas correspondientes:

Gráfico 01:



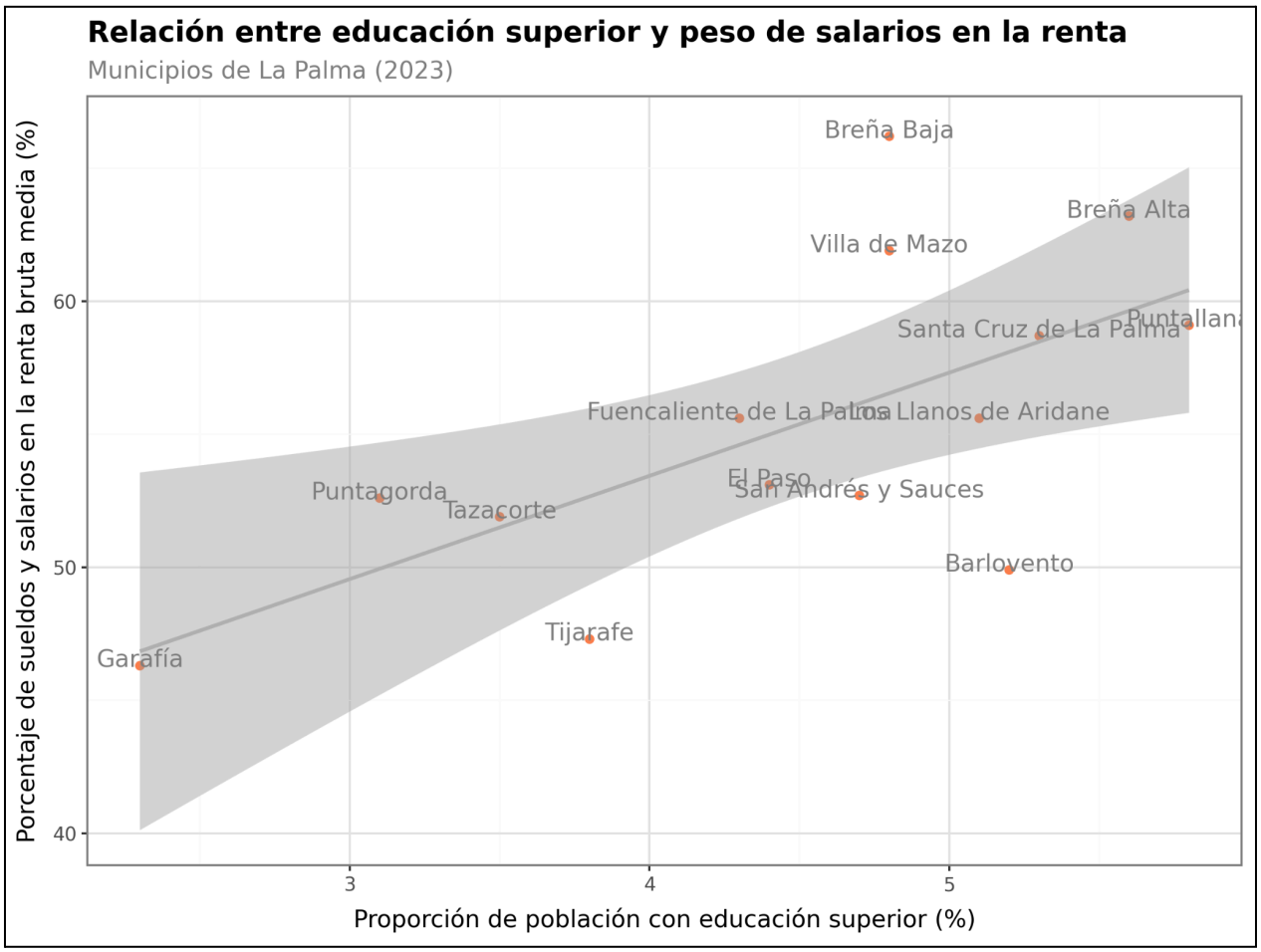
Link a gh-pages: [GitHub Pages: ia_graph_01_distribucion_renta.png]

Gráfico 02:



Link a gh-pages: [GitHub Pages: ia_graph_02_linea_gomera_sueldos_salarios.png]

Gráfico 03:



Link a gh-pages: [GitHub Pages: ia_graph_03_scatter_palma_salario_educación_2023.png]

Durante el proceso se identificaron diversas limitaciones en la generación automática de código. En múltiples ocasiones, el modelo no respetó completamente las restricciones definidas en los prompts. Se observaron respuestas que incluían sentencias de importación no solicitadas o desviaciones respecto a especificaciones concretas de diseño, como el número de columnas en los facets. Asimismo, el formateo de ejes en porcentaje no se aplicó de forma consistente, a pesar de haberse indicado explícitamente.

En el caso de la visualización de tipo scatter, se detectaron errores relacionados con los nombres de columnas, donde el modelo introdujo espacios adicionales que impedían la correcta ejecución del código. Ante esta situación, se optó por incorporar una fase de postprocesado que corrigiera automáticamente estas inconsistencias, en lugar de depender exclusivamente de ajustes en los prompts.

También se identificaron problemas derivados de la generación de código con sintaxis híbrida, en la que aparecían elementos propios de otros lenguajes como R, por ejemplo el uso de F en lugar de False. Este tipo de errores evidenció la necesidad de establecer mecanismos adicionales de validación antes de la ejecución. Por último, se observaron dificultades en el ajuste de parámetros de estilo, especialmente en el tamaño de los textos, lo que afectó a la legibilidad de algunas visualizaciones.

En conjunto, estos resultados pusieron de manifiesto que, aunque la generación de código mediante IA permite automatizar gran parte del proceso de visualización, resulta necesario complementar este enfoque con validaciones, restricciones adicionales y fases de corrección para garantizar la calidad final del sistema.

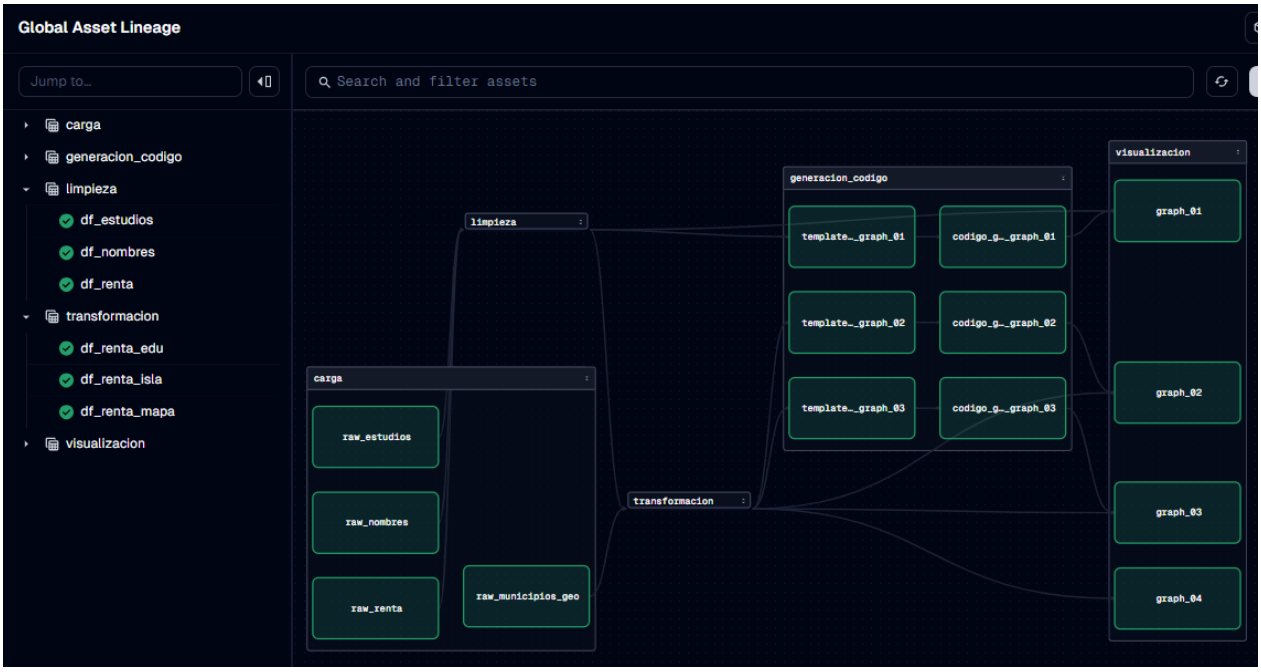
Visualización geoespacial de la distribución de la renta

En este apartado se abordó la construcción de una visualización geoespacial de la distribución de la renta a nivel municipal en Canarias. El objetivo consistió en representar la renta bruta media mediante un mapa coroplético, utilizando tanto herramientas programáticas basadas en Plotnine como una solución de inteligencia de negocio mediante Power BI.

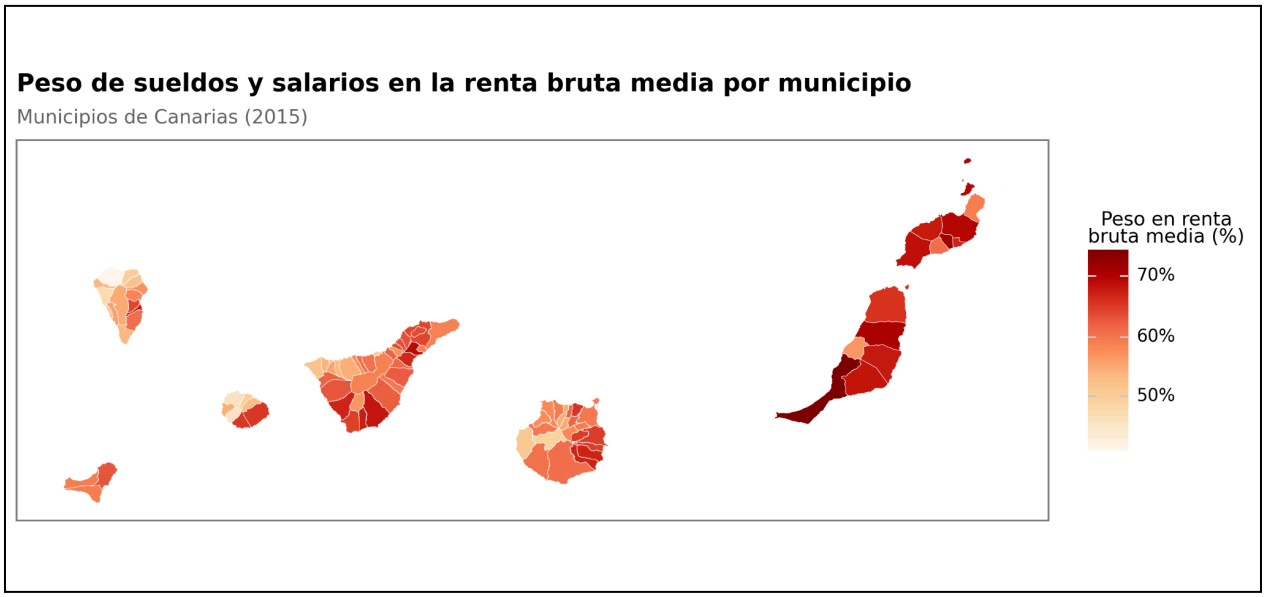
En primer lugar, se extendió el pipeline desarrollado previamente con la incorporación de nuevos assets orientados al tratamiento de información geográfica:

- raw_municipios_geo: Asset encargado de la carga del fichero GeoJSON que contiene la geometría de los municipios.
- df_renta_mapa: Asset de integración con df_renta. Une el GeoJSON municipal con el dataset de renta mediante el código territorial para generar la base cartográfica de rentas.
- graph_04: Asset de visualización, responsable de la generación del mapa. Se estableció como caso de estudio el año 2015, por tratarse del primer año disponible en el dataset, y la fuente de ingresos “Sueldos y salarios”, al ser la de mayor peso dentro de la distribución de la renta media en Canarias.

El pipeline completo, incluyendo los nuevos assets, correctamente materializado se vio así:



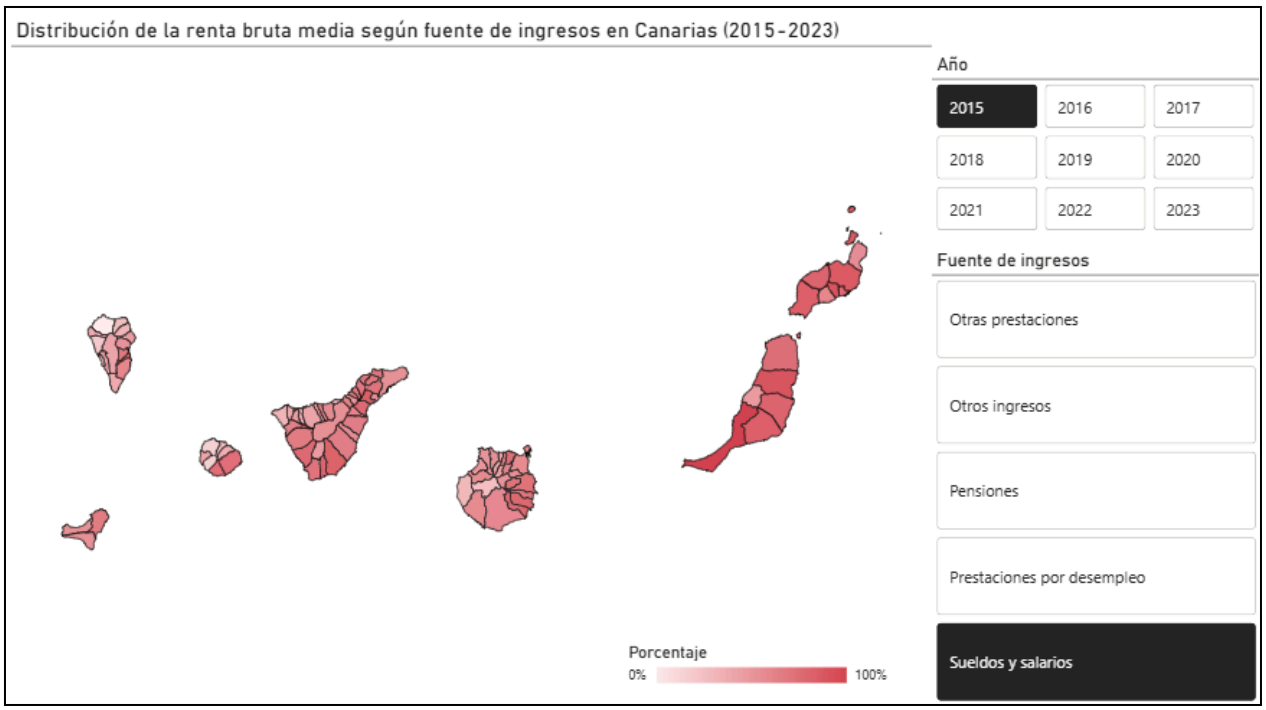
El resultado fue un mapa coroplético generado con Plotnine, en el que cada municipio se representó mediante su geometría y se coloreó en función del porcentaje adjudicado a sueldos y salarios en la renta bruta media. La visualización obtenida se incluye a continuación:



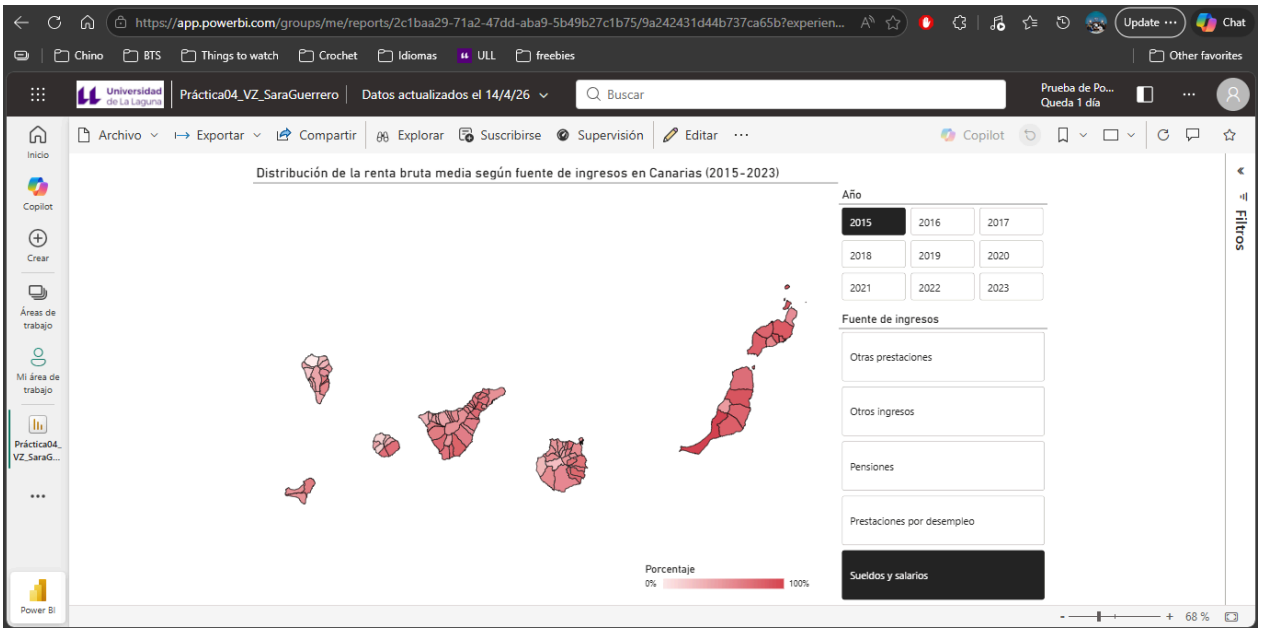
Link a gh-pages: [GitHub Pages: graph_04_mapa_municipios_salarios_2015.png]

De forma complementaria, se replicó la visualización en Power BI con el objetivo de comparar ambos enfoques. Para ello, se realizó previamente la conversión del fichero GeoJSON a formato TopoJSON, requisito necesario para su correcta carga en la herramienta. Posteriormente, se estableció la relación entre los datos de renta y la geometría mediante el código territorial, y se utilizó el porcentaje en la renta como variable para definir la intensidad del color en el mapa.

Además, se incorporaron elementos interactivos en forma de panel de control, que permitieron filtrar la información por año y por fuente de ingresos. Esta funcionalidad aportó una mayor flexibilidad en el análisis, ya que facilitó la exploración de distintas combinaciones sin necesidad de regenerar la visualización.



El informe resultante se publicó según los requisitos de la práctica:

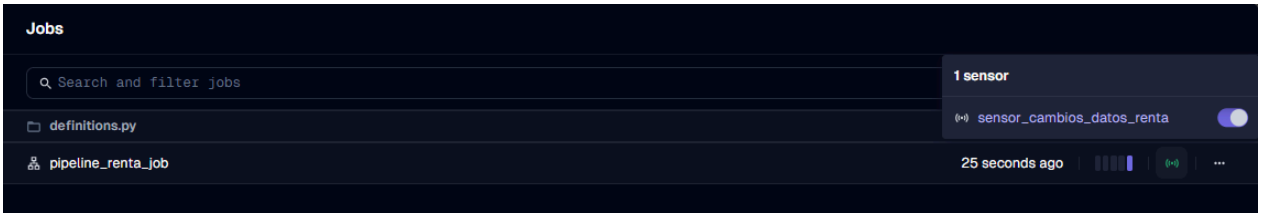


Link: https://app.powerbi.com/links/2PhJHgTVpu?ctid=328bd378-2b42-4021-9842-827ab6e83f4f&pbi_source=linkShare

Orquestación reactiva mediante sensor basado en eventos

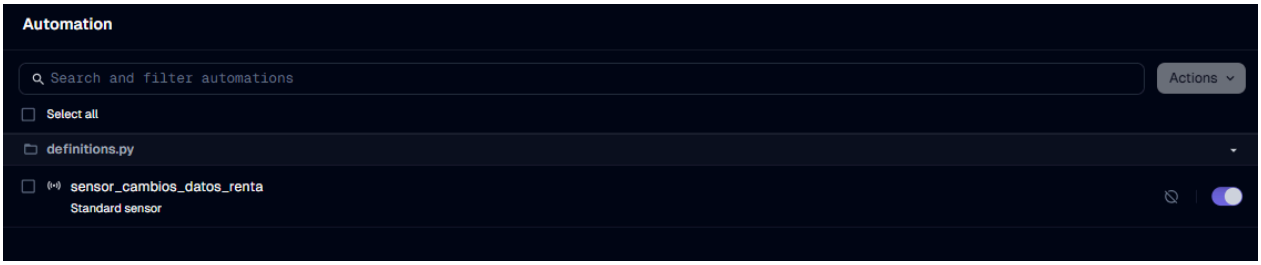
Con el objetivo de automatizar la ejecución del pipeline ante cambios en los datos de entrada, se implementó un mecanismo de orquestación reactiva basado en sensores en Dagster. En primer lugar, se definió un job que agrupa todos los assets asociados al pipeline de renta, lo que permitió ejecutar de forma conjunta las distintas fases del proceso, desde la carga de datos hasta la generación de visualizaciones.

```python
pipeline_renta_job = define_asset_job(
    name="pipeline_renta_job",
    selection=AssetSelection.all()
)
```



Sobre este job se implementó un sensor basado en eventos externos que monitoriza los cuatro ficheros de datos de entrada mediante su fecha de última modificación. Cada vez que el sensor detecta un cambio en alguno de estos archivos, lanza automáticamente la ejecución completa del pipeline, garantizando que las visualizaciones y resultados se mantengan actualizados sin intervención manual. De este modo, se introdujo un comportamiento reactivo en el sistema, alineado con principios de automatización y DataOps.

```python
@sensor(job=pipeline_renta_job)
def sensor_cambios_datos_renta(context):
    archivos_vigilados = [
        r"C:\Users\Usuario\Documents\Aaa ULL\Visualización\Practicas-Visualizacion-de-datos\Practica_02\data\distribucion-renta-canar
        r"C:\Users\Usuario\Documents\Aaa ULL\Visualización\Practicas-Visualizacion-de-datos\Practica_02\data\codislas.csv",
        r"C:\Users\Usuario\Documents\Aaa ULL\Visualización\Practicas-Visualizacion-de-datos\Practica_02\data\nivelestudios.xlsx",
        r"C:\Users\Usuario\Documents\Aaa ULL\Visualización\Practicas-Visualizacion-de-datos\Practica_04\data\Municipios-2024_geo.jsor
    ]

    estado_anterior = json.loads(context.cursor) if context.cursor else {}
    estado_actual = {}

    for ruta in archivos_vigilados:
        if os.path.exists(ruta):
            estado_actual[ruta] = os.path.getmtime(ruta)
        else:
            estado_actual[ruta] = None

    if estado_actual != estado_anterior:
        context.update_cursor(json.dumps(estado_actual, ensure_ascii=False))
        yield RunRequest(
            run_key=str(estado_actual),
            run_config={}
        )
    else:
        yield SkipReason("No hubo cambios en los archivos de datos.")
```



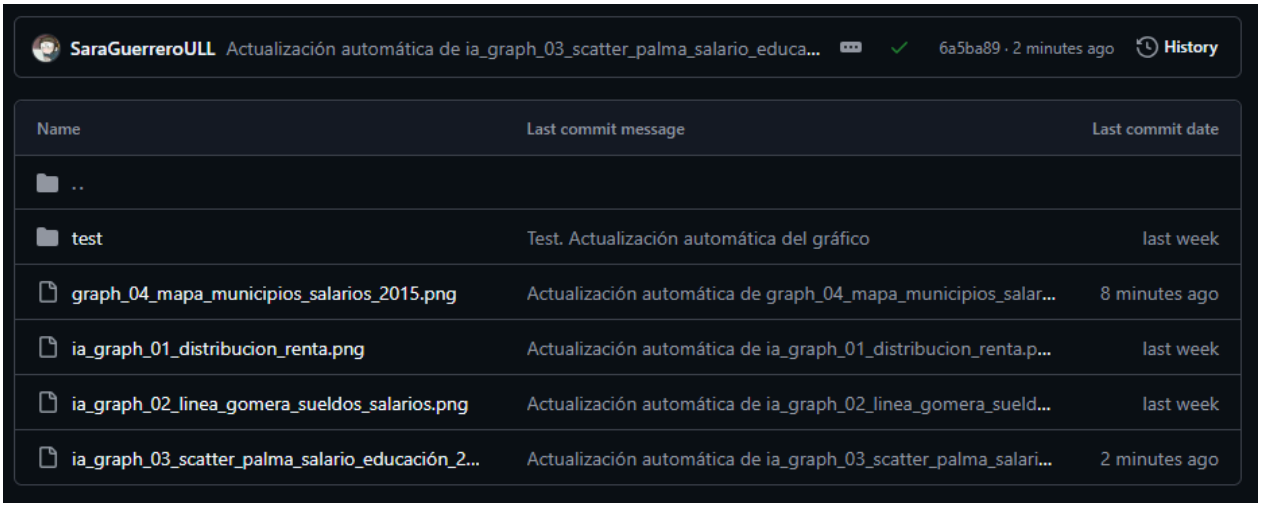
Control de versiones e integración continua

El proyecto se gestionó mediante Git como sistema de control de versiones, lo que permitió registrar de forma estructurada cada modificación realizada en el pipeline, los datos y las visualizaciones generadas. El repositorio se publicó en GitHub, lo que aseguró un respaldo remoto del trabajo y facilitó el seguimiento del historial de cambios.

Con el fin de automatizar la actualización de resultados, se incorporó en los assets de visualización un mecanismo para realizar commits de forma automática cada vez que se generaba una nueva imagen. Se usó el siguiente bloque de código adaptado a cada visualización:

```python
subprocess.run(["git", "add", path], check=False)
subprocess.run(
        ["git", "commit", "-m", "Actualización automática de ia_graph_01_distribucion_renta.png"],
        check=False
)
subprocess.run(["git", "push"], check=False)
```

En GitHub se logró comprobar los commits automáticos:



En el contexto de DataOps, la integración de Git y GitHub resultó clave para garantizar la trazabilidad, reproducibilidad e integridad del flujo de trabajo. Cada cambio quedó documentado de forma explícita, lo que facilitó la validación progresiva del desarrollo y aseguró una evolución controlada del proyecto. De este modo, la orquestación con Dagster se complementó con prácticas sólidas de control de versiones e integración continua.